# Looped Flows & Flow Map Language Models

## Cải thiện Suy luận Lặp và Mô hình hóa Ngôn ngữ Đơn bước

Hung Manh

October 26, 2026

# Nội dung trình bày


1. Dòng vòng lặp Looped Flows
2. Embedded Language Flows (ELF)
3. Mô hình ngôn ngữ ánh xạ dòng FMLM
4. Bộ nhớ dùng chung trong Transformer vòng lặp LPT

# Looped Flows: Tổng quan bài báo

- **Tên bài báo:** *Thinking with Looped Flows*
- **Tác giả và tổ chức:** Ayhan Suleymanzade, Chanhyuk Lee, Jinwoo Kim, Nicholas M. Boffi et al., EPFL, KAIST, CMU và các nơi khác, arXiv:2609.11801
- **Ý tưởng cốt lõi:** Huấn luyện mô hình vòng lặp bằng một chuỗi mục tiêu khử nhiễu cục bộ, thay vì lan truyền ngược qua nhiều bước lặp.
- **Đóng góp nổi bật:**
  - **Bộ khử nhiễu có trạng thái:** vừa khử nhiễu, vừa cập nhật trạng thái lặp. Nhiễu giảm dần và nhiễu dùng chung buộc trạng thái lặp mang thông tin hữu ích cho bước sau.
  - **Suy luận bằng tích phân dòng xác suất:** tăng tính toán qua lưới thời gian mịn hơn, và cho nhiều lời giải hợp lệ từ các mẫu nhiễu khác nhau.
  - **Kết quả:** ARC-AGI-1 đạt 58,8% (TRM 44,6%), ARC-AGI-2 đạt 12,2% (TRM 7,8%), Sudoku-Extreme đạt 97,9%, chỉ với 5 đến 7M tham số.

# 1. Looped Flows: Bối cảnh và thách thức

**Bối cảnh:** Bài toán khó cần nhiều tính toán hơn lúc suy luận. Mô hình vòng lặp cập nhật trạng thái ẩn nhiều lần để tích lũy tính toán.

**Ba khoảng trống của các hướng hiện có:**

- **Mô hình vòng lặp như HRM, TRM, FPRM:** lan truyền ngược qua toàn bộ chuỗi lặp vừa tốn bộ nhớ vừa bất ổn, nên phải chặn gradient. Mất mát ở bước $\geq i$ không gửi được tín hiệu cho trạng thái $z_{<i}$.
- **Mô hình dòng và khuếch tán:** học khử nhiễu cục bộ, không cần lan truyền ngược qua quỹ đạo, nhưng không có trạng thái lặp, nên yếu với tính toán tuần tự.
- **Mô hình vòng lặp ngẫu nhiên như PTRM, EqR, GRAM:** thêm tính ngẫu nhiên nhưng vẫn dựa vào gradient cục bộ để xây trạng thái lặp hữu ích toàn cục.

**Hậu quả:** chuỗi lặp bất ổn, rơi vào điểm hút giả hoặc không hội tụ, ngay cả ở bài toán đơn giản. Hướng giải quyết: kết hợp mục tiêu khử nhiễu cục bộ với trạng thái lặp.

# 2. Looped Flows: Kiến thức nền

**Mô hình vòng lặp:** $z_i = f(z_{i-1}; c)$, đầu đọc $\hat{x}_i = g(z_i)$. Thực tế huấn luyện với chặn gradient $\mathrm{sg}$:

$$\mathcal{L}_{\text{loop}} = \mathbb{E}_{c,x}\Big[\sum_i \mathrm{CE}(\hat{x}_i, x)\Big], \qquad z_i = f\big(\mathrm{sg}(z_{i-1}); c\big)$$

- Điểm yếu: mất mát ở bước $\geq i$ không truyền được về $z_{<i}$. Quá trình lặp xác định chỉ cho một lời giải.

**Khớp dòng cho dữ liệu phân loại:** nội suy $I_t = (1-t)x_0 + tx_1$, với $x_0 \sim \mathcal{N}(0, \sigma^2 I)$.

- Bộ khử nhiễu $D_t(x; c) = \mathbb{E}[x_1 \mid I_t = x, c]$, vận tốc $b_t(x; c) = \frac{D_t(x;c) - x}{1-t}$.
- Hàm mục tiêu $\mathcal{L}_{\text{flow}} = \mathbb{E}\left[\mathrm{CE}(\hat{x}_t, x_1)\right]$ với $t \sim \mathcal{U}[0, 1]$: mất mát tại $t$ không phụ thuộc tính toán ở $t' < t$, nên không cần lan truyền ngược qua quỹ đạo.
- Điểm yếu: không có trạng thái, khó học tính toán tuần tự.

# 3. Looped Flows: Bộ khử nhiễu có trạng thái

**Ý tưởng:** Biến bộ khử nhiễu thành có trạng thái, vừa khử nhiễu trạng thái dòng $x_{t_i}$, vừa cập nhật trạng thái lặp $z_{t_i}$:

$$(\hat{x}_{t_i}, z_{t_{i+1}}) = \hat{D}_{t_i}(x_{t_i}, z_{t_i}; c)$$

- Suy luận dùng bước Euler tiến trên lưới thời gian $0 = t_0 < \cdots < t_n = 1$:

$$x_{t_{i+1}} = x_{t_i} + (t_{i+1} - t_i)\,\frac{\hat{x}_{t_i} - x_{t_i}}{1 - t_i}$$

- **Kiến trúc:** xây trên TRM, thêm lớp chiếu cho nội suy nhiễu và nhúng thời gian, thêm khoảng 5% tham số. Trạng thái lặp gồm hai thành phần, đặc trưng ẩn $h$ để giải mã dự đoán và thành phần $\ell$ để cập nhật $h$. Chỉ lan truyền ngược qua chu kỳ cuối của mỗi lần gọi.
- **Quy mô:** 5M tham số cho Sudoku với MLP-Mixer, 7M tham số cho các tác vụ còn lại với Transformer.

# 4. Looped Flows: Căn chỉnh theo thời gian

**Hàm mục tiêu:** lấy $k+1$ mốc thời gian đều, sắp tăng dần $t_0 < \cdots < t_k$, rồi cộng cross-entropy qua các bước:

$$\mathcal{L}_{\mathrm{LF}} = \mathbb{E}\Big[\sum_{i=0}^{k-1} \mathrm{CE}(\hat{x}_{t_i}, x_1)\Big], \quad (\hat{x}_{t_i}, z_{t_{i+1}}) = \hat{D}_{t_i}\big(I_{t_i}, \mathrm{sg}(z_{t_i}); c\big)$$

**Hai thiết kế then chốt:**

- **Nhiễu giảm dần:** mức nhiễu $1 - t_i$ giảm theo thời gian, thành chuỗi bài khử nhiễu từ khó đến dễ.
- **Nhiễu dùng chung:** cùng một cặp $(x_0, x_1)$ cho mọi $t_i$ trong một lần chạy. Các bước kề nhau có đầu vào gần nhau, nên trạng thái lặp $z_{t_i}$ vẫn hữu ích ở bước sau dù gradient chỉ phủ một bước.

**Mục tiêu giả:** khi dữ liệu ít, ví dụ 1000 mẫu Sudoku, nội suy ở $t$ lớn gần như lộ $x_1$ và gây overfit. Thay $x_1$ trong nội suy bằng dự đoán trước đó $\mathrm{sg}(\hat{x}_{t_{i-1}})$, xác suất dùng tăng tuyến tính từ 0 đến 1 trong 20 nghìn bước. Mất mát vẫn dùng $x_1$ thật.

# 5. Looped Flows: Thời gian tính toán thích ứng và cấu hình

- **Vấn đề:** nhiễu giảm dần làm mất mát khử nhiễu giảm đơn điệu và bão hòa, các bước cuối không còn tín hiệu học.
- **Đầu dừng** $q$ là bộ phân loại nhị phân xem dự đoán hiện tại đã đúng chưa:

$$\mathcal{L}_{\text{ACT}} = \mathbb{E}\Big[\sum_i \lambda \,\text{BCE}\big(q(z_{t_{i+1}}), \mathbb{1}(x_1 = \text{round}(\hat{x}_{t_i}))\big)\Big]$$

  Khi $q > 1/2$, các bước sau bị bỏ qua lúc huấn luyện. Tổng mất mát $= \mathcal{L}_{\text{LF}} + \mathcal{L}_{\text{ACT}}$, $\lambda = 0{,}5$.
- **Cấu hình:** $k = 16$ bước, Adam-atan2, tốc độ học đỉnh $10^{-4}$, batch 768, EMA 0,999, cắt gradient 1,0.
- **Tài nguyên:** khoảng 1 đến 5 giờ cho Sudoku, Maze, N-Queens, Graph Coloring. Khoảng 1 đến 2 ngày cho ARC.

# 6. Looped Flows: Cơ chế suy luận

- **Tích phân xác định bằng ODE:** bước Euler tiến, mỗi lần gọi bộ khử nhiễu cập nhật cả $x$ và $z$.
- **Tích phân ngẫu nhiên bằng SDE:** tương đương SDE của dòng xác suất, thực hiện bằng lùi nhiễu. Mỗi bước đẩy $x_{t_i}$ lùi về thời điểm $s = a\,t_i$, thêm nhiễu mới, rồi bước Euler tới $t_{i+1}$. Hệ số $\gamma = 0$ quay về ODE, $\gamma > 0$ thêm tính ngẫu nhiên.
- **Mở rộng tính toán lúc suy luận:**
  - **Lưới thời gian mịn hơn lúc huấn luyện.** Sudoku huấn luyện 16 bước nhưng suy luận 128 bước.
  - **Tổ hợp chọn mẫu theo điểm Q cao nhất:** chạy nhiều quỹ đạo độc lập, thường 5, chọn dự đoán có xác suất thành công cao nhất theo đầu dừng $q$.
- **Nhiều lời giải hợp lệ:** xuất phát từ các mẫu nhiễu $x_0$ khác nhau, vận chuyển xác suất tới các mode khác nhau của $p_{\text{data}}(x \mid c)$. Mô hình vòng lặp xác định chỉ cho một lời giải mỗi bài.

# 7. Looped Flows: Kết quả trên các bài toán suy luận

**Độ chính xác lời giải, phần trăm. ARC dùng pass@2. Looped Flows chạy ba hạt giống.**

| Phương pháp | Sudoku-Extreme | Maze-Hard | ARC-AGI-1 | ARC-AG |
|---|---|---|---|---|
| HRM, 27M | 55,0 | 74,5 | 40,3 | 5,0 |
| TRM, 5 đến 7M | 87,4 | 85,3 | 44,6 | 7,8 |
| FPRM, 7M | 94,2 | **87,0** | 47,5 | 6,2 |
| GRAM, 10M | – | – | 52,0 | 11,1 |
| **Looped Flows, 5 đến 7M** | **97,9 ± 0,4** | 86,7 ± 1,1 | **58,8 ± 1,8** | **12,2** ± |

- Tốt nhất ở 3 trong 4 bài; Maze-Hard chỉ ngang FPRM, bài báo không giải thích.
- Tổ hợp 5 quỹ đạo: Sudoku 99,3, Maze 86,9, ARC-AGI-1 59,5, ARC-AGI-2 12,2. Cạnh tranh với PTRM dùng 100 quỹ đạo và EqR dùng 128 quỹ đạo.
- Độ lệch chuẩn trên ARC khoảng 1,8 đến 1,9 điểm: kết quả nhạy với hạt giống.

# 8. Looped Flows: Mở rộng lúc suy luận và độ ổn định

**Tăng số bước tích phân trên Sudoku:**

| **Số bước** | 8 | 16 | 32 | 64 | 128 |
|---|---|---|---|---|---|
| **Độ chính xác (%)** | 74,5 | 85,2 | 93,0 | 96,7 | 97,9 |

- Vượt GRAM tại 32 bước.

**Độ ổn định của chuỗi lặp, trên khoảng 65 nghìn bài Sudoku-Extreme:**

- TRM thất bại ở 12,6% số bài: 88,3% do không hội tụ, 11,7% do điểm hút giả.
- Looped Flows cứu được **90,9%** các bài TRM thất bại: 89,9% trường hợp không hội tụ và 98,0% trường hợp điểm hút giả.

# 9. Looped Flows: Bài toán đa lời giải

**So sánh với GRAM. Độ phủ là tỉ lệ lời giải thật được tìm thấy.**

| Bài toán | Chỉ số | GRAM | Looped Flows |
|---|---|---|---|
| N-Queens 8×8 | Độ chính xác | 99,7 | **99,9** |
| | Độ phủ | 90,3 | **91,4** |
| N-Queens 10×10 | Độ chính xác | 96,1 | 94,4 |
| | Độ phủ | **74,3** | 61,5 |
| Tô màu đồ thị 8 đỉnh | Số xung đột ↓ | 2,7 | **0,7** |
| | Độ phủ | 85,8 | **89,4** |
| Tô màu đồ thị 10 đỉnh | Số xung đột ↓ | 3,3 | **1,0** |
| | Độ phủ | 51,3 | **55,2** |

- Trên N-Queens 10×10, GRAM vẫn hơn ở cả độ chính xác và độ phủ. Looped Flows tốt nhất ở 6 trong 8 chỉ số.

# 10. Looped Flows: Nghiên cứu loại bỏ thành phần

**Khi huấn luyện, trên ARC-AGI-1 và ARC-AGI-2:**

| Cấu hình | ARC-AGI-1 | ARC-AGI-2 |
|---|---|---|
| **Đầy đủ** | **58,8** | **12,2** |
| Bỏ điều kiện thời gian | 56,4 | 9,9 |
| Bỏ nội suy | 51,5 | 9,9 |
| Bỏ cả hai | 43,6 | 5,0 |
| Bỏ nhiễu giảm dần | 51,6 | 9,9 |
| Bỏ nhiễu dùng chung | 56,4 | 10,8 |

- Nhiễu giảm dần quan trọng nhất; nhiễu dùng chung ít hơn nhưng vẫn giúp. Bỏ cả hai thành phần dòng thì ngang TRM (43,6 so với 44,6).
- **Lúc suy luận:** SDE với $\gamma = 5$ nhất quán tốt hơn ODE, ví dụ Sudoku 97,9 so với 97,6, ARC-AGI-1 58,8 so với 57,5.
- **Trên Sudoku:** dòng không vòng lặp ghi nhớ dữ liệu, tổng quát hóa kém; tự điều kiện hóa tốt hơn; Looped Flows tốt nhất.

# 11. Looped Flows: Hạn chế, phản biện và kết luận

**Hạn chế và điểm cần thận trọng:**

- Chưa có lý thuyết cho căn chỉnh theo thời gian. Nhiễu dùng chung về lý thuyết cho phép đường tắt triệt nhiễu; bài báo chỉ cho thấy gián tiếp rằng mô hình không dùng nó.
- Lệch phân phối giữa huấn luyện dùng nội suy $I_{t_i}$ và suy luận dùng $x_{t_i}$ từ bước Euler trước. Bài báo không bàn tới.
- Mô hình 5 đến 10M tham số, chỉ trên câu đố suy luận, nhiều siêu tham số riêng cho từng tác vụ. Nghiên cứu loại bỏ chỉ trên ARC.
- Huấn luyện vẫn cần chạy tuần tự $k$ bước, chưa phải huấn luyện không mô phỏng.

**Kết luận:**

- Mục tiêu khử nhiễu cục bộ cộng căn chỉnh theo thời gian huấn luyện được trạng thái lặp ổn định, kể cả khi gradient chỉ phủ một bước.
- Cho phép mở rộng tính toán lúc suy luận và sinh nhiều lời giải. **Hướng tiếp theo:** huấn luyện không mô phỏng, quy mô lớn hơn.

# ELF: Tổng quan bài báo

- **Tên bài báo:** *Embedded Language Flows (ELF)*
- **Ý tưởng cốt lõi:** Mô hình khuếch tán ngôn ngữ liên tục dựa trên Flow Matching trong không gian nhúng biểu diễn (latent space).
- **Đóng góp chính:**
  - Chứng minh continuous diffusion hoàn toàn vượt trội discrete diffusion trên ngôn ngữ nếu khử nhiễu hoàn toàn trong không gian nhúng liên tục.
  - Chỉ thực hiện rời rạc hóa (discretization/decoding) tại bước cuối cùng ($t = 1$).
  - Sử dụng mạng chia sẻ trọng số giữa denoising và decoding, tích hợp sẵn Classifier-Free Guidance (CFG) lúc huấn luyện.

# 1. ELF: Bối cảnh & Phân loại DLMs

**Bối cảnh:**

- Diffusion và flow-based models là tiêu chuẩn vàng trong sinh ảnh/video, đang được mở rộng sang xử lý ngôn ngữ.

**Phân loại mô hình khuếch tán ngôn ngữ (DLMs):**

- *Rời rạc (Discrete):* Thao tác trực tiếp trên token (trước đây được ưu tiên do dễ tối ưu hóa).
- *Liên tục (Continuous):* Ánh xạ token sang không gian biểu diễn liên tục để tiến hành khử nhiễu.

**Giải pháp đề xuất của ELF:**

- Khử nhiễu hoàn toàn trong không gian nhúng liên tục.
- Định nghĩa trường vận tốc qua thời gian liên tục với Flow Matching.
- Dùng mạng chia sẻ trọng số, chỉ rời rạc hóa tại bước $t = 1$.

# 2. ELF: Khung làm việc (Framework)

- **Từ rời rạc sang liên tục:** Chuỗi token $s \in \mathcal{V}^L$ được chuyển sang biểu diễn liên tục $x$ bằng bộ mã hóa T5 đông cứng.
- **Nội suy tuyến tính (Rectified Flows):** Tạo biến ẩn $z_t$ dọc theo đường đi:

$$z_t = tx + (1-t)\epsilon, \quad t \in [0,1], \quad \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

- **Trường vận tốc dòng (Flow Velocity):** Đạo hàm theo thời gian:

$$v = \frac{dz}{dt} = x - \epsilon$$

- **Dự đoán $x$ (x-prediction):** Trực tiếp dự đoán biểu diễn sạch $x_\theta(z_t, t)$ thay vì dự đoán vận tốc $v$, giúp mạng nơ-ron hoạt động ổn định ở số chiều cực cao.

# 3. ELF: Hàm mục tiêu & Rời rạc hóa

- **Nhánh khử nhiễu (Denoising - chiếm 80% trọng số):** Tối thiểu hóa sai số MSE của trường vận tốc dự đoán:

$$\mathcal{L}_{\text{MSE}} = \mathbb{E}_{t,x,\epsilon}\left[\frac{1}{(1-t)^2}\|x_\theta(z_t,t) - x\|^2\right]$$

- **Nhánh giải mã (Decoding - chiếm 20% trọng số):**
  - Tại bước cuối $t = 1$, biến đổi $z_t$ thành phân phối token qua ma trận unembedding $W$.
  - Dùng chung trọng số với mạng khử nhiễu, huấn luyện bằng Cross-Entropy:

$$\mathcal{L}_{\text{CE}} = \mathbb{E}_{\tilde{z}}\left[\text{CrossEntropy}(Wx_\theta(\tilde{z}), s)\right]$$

# 4. ELF: Điều kiện hóa & Dẫn hướng (Guidance)

- **Self-conditioning:** Thực hiện lượt truyền thứ hai điều kiện hóa trên dự đoán trung gian $\hat{x}'$ từ bước trước để tăng độ mượt của quỹ đạo.
- **Classifier-Free Guidance (CFG):** Dễ dàng áp dụng trong miền liên tục:

$$v_{\text{cfg}}(z_t \mid c) = \omega v(z_t \mid c) + (1 - \omega) v(z_t \mid \emptyset)$$

- **Training-time CFG:** Tích hợp thẳng CFG vào hàm mục tiêu vận tốc ngay từ lúc huấn luyện $\rightarrow$ loại bỏ hoàn toàn chi phí tính toán kép khi suy luận.
- **Sinh có điều kiện:** Ghép prompt vào trước chuỗi mục tiêu; phần prompt được giữ nguyên không làm nhiễu.

# 5. ELF: Kết quả thực nghiệm

**Sinh không điều kiện (OpenWebText):**

- Ở 32 bước lấy mẫu, ELF-B (105M) đạt **Gen. PPL 24.08**, vượt trội so với các đối thủ rời rạc như MDLM (104.29) và Duo (62.47).
- **Hiệu quả dữ liệu cực cao:** Đạt đỉnh chỉ với 45B token huấn luyện (ít hơn $\sim$10 lần so với $> 500$B tokens của các baseline).

**Chưng cất lũy tiến (Progressive Distillation):**

- Cho phép giảm số bước lấy mẫu xuống mức cực ít (**1 đến 4 bước**) mà vẫn duy trì chất lượng sinh văn bản xuất sắc.

**Sinh có điều kiện:**

- **Tóm tắt XSum:** ROUGE-1 đạt **36.0** (đánh bại các mô hình tự hồi quy cùng kích cỡ).
- **Dịch máy WMT14:** Đạt BLEU **26.4**.

# 6. ELF: Kết luận & Đóng góp

- **Khẳng định:** Mô hình khuếch tán ngôn ngữ liên tục hoàn toàn có thể cạnh tranh và vượt trội hơn các phương pháp rời rạc.
- **Bí quyết tối ưu của ELF:**
  1. Giữ toàn bộ quỹ đạo làm nhiễu trong không gian nhúng liên tục.
  2. Chỉ thực hiện rời rạc hóa ở bước thời gian cuối cùng ($t = 1$).
  3. Sử dụng mạng chia sẻ trọng số, loại bỏ bộ giải mã phụ trợ rườm rà.
- **Tác động:** Mở ra tiềm năng tạo văn bản chất lượng cao với chi phí huấn luyện thấp và tốc độ suy luận cực nhanh.

# FMLM: Tổng quan bài báo

- **Tên bài báo:** *Flow Map Language Models: One-step Language Modeling via Continuous Denoising*
- **Tác giả và tổ chức:** Chanhyuk Lee, Nicholas M. Boffi, Jinwoo Kim et al., KAIST và CMU, arXiv:2602.16813
- **Ý tưởng cốt lõi:** Chạy dòng liên tục trên biểu diễn one-hot của token, rồi chưng cất thành ánh xạ dòng để sinh văn bản chỉ với một bước.
- **Đóng góp nổi bật:**
  - **Phản bác định kiến:** Dòng liên tục cạnh tranh được với khuếch tán rời rạc ngay cả trên dữ liệu rời rạc.
  - **Bộ khử nhiễu hai thời điểm $\delta_{s,t}$:** Luôn nằm trên đơn hình xác suất, nên huấn luyện ánh xạ dòng bằng KL và cross-entropy thay vì sai số bình phương.
  - **Sinh một bước:** Gen. PPL 119,34 trên LM1B, trong khi các mô hình khuếch tán rời rạc đã chưng cất vượt 1000 ở một bước.

# 1. FMLM: Lỗi phân rã nhân tử của khuếch tán rời rạc

**Nút thắt của mô hình tự hồi quy:** Sinh tuần tự từng token, cần $L$ lần gọi mạng cho chuỗi dài $L$.

**Khuếch tán rời rạc buộc phải xấp xỉ:**

$$\tilde{p}_{t|s}(\mathbf{y}_t \mid \mathbf{y}_s) := \prod_{l=1}^{L} p^l_{t|s}(y^l_t \mid \mathbf{y}_s)$$

- Giả định các token độc lập có điều kiện, chỉ đúng khi bước nhảy $t \to s$ rất nhỏ.
- Ví dụ đồ chơi với hai mode “new-york” và “san-diego”: ít bước sẽ sinh ra cả “new-diego” và “san-york”.
- Hậu quả ở 1 đến 4 bước: văn bản vô nghĩa với Gen. PPL trên 1000, hoặc lặp token với entropy dưới 4.

**Mệnh đề 3.5:** Trên tập hữu hạn, tồn tại phân phối đích không thể thu được từ phân phối nguồn bằng một ánh xạ xác định ở mức mẫu. Vì vậy khuếch tán rời rạc không có ánh xạ dòng ở mức mẫu.

# 2. FMLM: Mô hình dòng FLM trên biểu diễn one-hot

- **Biểu diễn:** $f(\mathbf{y}) \in \mathbb{R}^{L\times|\mathcal{V}|}$ là one-hot từng token, giải mã bằng argmax. Không cần embedding học.
- **Nội suy tuyến tính** giữa nhiễu Gaussian $\mathbf{x}_0$ và dữ liệu one-hot $\mathbf{x}_1$:

$$I_t = (1-t)\mathbf{x}_0 + t\mathbf{x}_1, \qquad \mathbf{b}_t(\mathbf{x}) = \frac{D_t(\mathbf{x}) - \mathbf{x}}{1-t}$$

- **Học vận tốc bằng sai số bình phương thất bại:** đích $\mathbf{x}_1 - \mathbf{x}_0$ có hạng đầy đủ, mạng bị underfit khi $|\mathcal{V}| \gg d$.
- **Học bộ khử nhiễu thay thế:** $D_t(\mathbf{x}) = \mathbb{E}[\mathbf{x}_1 \mid I_t = \mathbf{x}]$ chính là phân phối hậu nghiệm của từng token, nằm trên đơn hình xác suất (Bổ đề 3.1).
- **Huấn luyện bằng cross-entropy** với đầu ra softmax theo token. Mệnh đề 3.2: nghiệm tối ưu duy nhất là $D_t$, và sai số nhỏ kéo theo khoảng cách Wasserstein $W_2$ nhỏ.

# 3. FMLM: Tái tham số hóa thời gian theo tỉ lệ lỗi giải mã

- **Vấn đề:** Tỉ lệ lỗi giải mã $P_e(t)$ giảm rất gấp gần $t = 1$ khi $|\mathcal{V}|$ lớn. Lấy $t$ đều làm lãng phí tín hiệu huấn luyện và lãng phí bước lấy mẫu.

$$P_e(t) = \frac{1}{L} \sum_{l=1}^{L} \mathbb{P}\big(g^l(\mathbf{x}_t) \neq g^l(\mathbf{x}_1)\big)$$

- **Đề xuất:** Huấn luyện và lấy mẫu đều theo $\tau(t)$, sao cho mỗi bước giảm cùng một lượng lỗi giải mã:

$$\tau(t) = 1 - \frac{|\mathcal{V}|}{|\mathcal{V}| - 1} P_e(t)$$

- **Cài đặt:** Bảng tra 1000 điểm tính bằng cầu phương Gauss–Hermite, nội suy spline bậc ba. Chi phí gần như bằng không.
- **Hiệu quả:** Cho phép mở rộng tới $|\mathcal{V}| \approx 50\,000$. Trên LM1B, Gen. PPL của FLM: không tái tham số hóa 149,18; thời gian entropy học được 130,42; theo tỉ lệ lỗi giải mã **106,98**.

# 4. FMLM: Bộ khử nhiễu hai thời điểm trên đơn hình xác suất

**Ánh xạ dòng:** $X_{s,t}(\mathbf{x}) = \mathbf{x} + (t-s)\,\mathbf{v}_{s,t}(\mathbf{x})$, nhảy thẳng giữa hai thời điểm bất kỳ. Sinh một bước: $\hat{\mathbf{x}}_1 = X_{0,1}(\mathbf{x}_0)$.

- *Khó khăn:* $X_{s,t}$ rời khỏi đơn hình do nhiễu Gaussian, nên khó huấn luyện bằng cross-entropy.

**Định nghĩa:** $\delta_{s,t}(\mathbf{x}) := \mathbf{x} + (1-s)\,\mathbf{v}_{s,t}(\mathbf{x})$. Mệnh đề 3.3:

1. **Khôi phục ánh xạ dòng:** $X_{s,t}(\mathbf{x}) = \frac{1-t}{1-s}\mathbf{x} + \frac{t-s}{1-s}\,\delta_{s,t}(\mathbf{x})$.
2. **Nằm trên đơn hình:** $\delta_{s,t}(\mathbf{x})^l \in \Delta^{|\mathcal{V}|-1}$, nên dùng được softmax.
3. **Trùng bộ khử nhiễu trên đường chéo:** $\delta_{s,s} = D_s$.
4. **Điều kiện nửa nhóm dạng tổ hợp lồi:** $\delta_{s,t} = \gamma\,\delta_{s,u} + (1-\gamma)\,\delta_{u,t}(X_{s,u})$, với $\gamma = \frac{(1-t)(u-s)}{(1-u)(t-s)} \in [0,1]$.

# 5. FMLM: Chưng cất nửa nhóm và sinh một bước

**Hàm mục tiêu KL nửa nhóm** với giáo viên FLM đóng băng $\hat{D}$ (Mệnh đề 3.4):

$$\mathcal{L}_{\mathrm{KL}} = \mathbb{E}\left[\sum_{l} \mathrm{KL}\big(\bar{\delta}^{l}_{s,t} \,\|\, \hat{\delta}^{l}_{s,t}(I_s)\big)\right] + \mathbb{E}\left[\sum_{l} \mathrm{KL}\big(\hat{D}^{l}_{t}(I_t) \,\|\, \hat{\delta}^{l}_{t,t}(I_t)\big)\right]$$

với mục tiêu giáo viên $\bar{\delta}_{s,t} = \mathrm{sg}\big(\gamma\, \hat{\delta}_{s,u}(I_s) + (1-\gamma)\, \hat{\delta}_{u,t}(\hat{X}_{s,u})\big)$.

- **Lấy mẫu bộ ba** $(s,u,t)$**:** chọn độ dài bước $h$, điểm đầu $\tau(s)$, điểm cuối $\tau(t) = \tau(s) + h$, điểm giữa $u$ là trung điểm. Cách này tốt nhất trong các cách thử.
- **Lấy mẫu biên:** cặp $(s,t) = (0,1)$ với xác suất $1/32$, vì $\tau$ phẳng gần $t = 0$. Bỏ kỹ thuật này, Gen. PPL một bước tăng từ 119,34 lên 142,61.
- **Cấu hình:** chưng cất 100 nghìn bước, chia đôi batch cho đường chéo và ngoài đường chéo. Lấy mẫu dùng thuật toán $\gamma$ của Kim et al.

# 6. FMLM: Kết quả của FLM ở 1024 bước

**Gen. PPL theo GPT-2 Large, 1024 bước. Entropy của dữ liệu: LM1B 4,31; OWT 5,44.**

| Mô hình | LM1B PPL | LM1B Ent | OWT PPL | OWT Ent |
|---|---|---|---|---|
| RDLM | 268,21 | 4,33 | – | – |
| CANDI | 120,99 | 4,35 | 143,13 | 5,71 |
| MDLM | 109,21 | 4,32 | 105,15 | 5,63 |
| Duo | 98,14 | 4,31 | 77,69 | 5,55 |
| **FLM** | **96,91** | 4,29 | **62,23** | 5,33 |

- Trên LM1B, FLM chỉ hơn Duo khoảng 1,3%. Trên OWT, FLM hơn rõ nhưng entropy thấp hơn dữ liệu, tác giả tự thừa nhận có đánh đổi nhỏ.
- Trên OWT, các đối chứng dùng checkpoint công bố chính thức, không huấn luyện lại cùng cấu hình.
- Mở rộng cỡ mô hình trên OWT, 150 nghìn bước: 179M, 424M, 870M cho Gen. PPL 75,57, 65,94, 61,59.

# 7. FMLM: Kết quả sinh một đến bốn bước

**Gen. PPL, thấp hơn là tốt hơn. Dấu sao đỏ: entropy dưới 4,0, tức lặp token.**

| Tập, số bước | MDLM+SDTT | MDLM+Di4C | Duo+DCD | Duo+Di4C | FMLM |
|---|---|---|---|---|---|
| LM1B, 1 bước | 1429,48 | 1217,10 | 1224,52 | 292,94* | **119,34** |
| LM1B, 2 bước | 602,14 | 621,59 | 520,08 | 247,69* | **110,19** |
| LM1B, 4 bước | 241,01 | 247,32 | 210,88 | 150,67 | **98,76** |
| OWT, 1 bước | 1260,86 | 1298,80 | 5743,29 | 370,51* | **168,30** |
| OWT, 2 bước | 877,22 | 758,23 | 891,16 | 210,22 | **133,29** |
| OWT, 4 bước | 339,73 | 239,27 | 250,86 | 154,67 | **111,31** |

- Entropy của FMLM một bước: LM1B 4,16 và OWT 5,17, thấp hơn dữ liệu thật (4,31 và 5,44).
- Một bước của FMLM tương đương các mô hình đối chứng ở 8 đến 16 bước trên LM1B và 4 đến 8 bước trên OWT. Hình 1 báo tăng tốc 8,3 lần.

# 8. FMLM: Dẫn hướng lúc suy luận

**Tự dẫn hướng** ngoại suy vận tốc ra xa mô hình yếu, mô hình yếu là bản có dropout 0,1:

$$\hat{\mathbf{b}}_t^{\text{guided}} = \hat{\mathbf{b}}_t^{\text{weak}} + \eta\big(\hat{\mathbf{b}}_t - \hat{\mathbf{b}}_t^{\text{weak}}\big)$$

- FLM ổn định đến $\eta = 100$. Tại $\eta = 50$, Gen. PPL giảm từ 96,91 xuống **51,62**, nhưng entropy giảm còn 4,00.
- Duo và MDLM sụp đổ từ $\eta \geq 10$.

**Dẫn hướng theo phần thưởng** dựa trên nhìn trước điểm cuối của ánh xạ dòng:

$$\mathbf{x}_{t_{n+1}} = X_{t_n,t_{n+1}}(\mathbf{x}_{t_n}) + \lambda\, \nabla_{\mathbf{x}_{t_n}} r\big(X_{t_n,1}(\mathbf{x}_{t_n})\big)$$

- Bộ phân loại phần thưởng chỉ cần huấn luyện trên văn bản sạch. Khuếch tán rời rạc phải huấn luyện trên dữ liệu bị nhiễu ở mọi mức.
- Ở 2 bước, chủ đề: FMLM đạt PPL 131,8 và phần thưởng 0,670; MDLM+SDTT đạt 888,8 và 0,274. Tiêu chí ngữ pháp: phần thưởng của mọi phương pháp đều thấp.

# 9. FMLM: Ràng buộc logic trên Sudoku và độ đa dạng

**Tỉ lệ lời giải hợp lệ, phần trăm, ở 1, 2, 4 và 1024 bước:**

| | Sinh không điều kiện | Giải puzzle, tập kiểm tra |
|---|---|---|
| MDLM+SDTT | 0 / 0 / 0,02 / 92,19 | 0 / 2,93 / 8,40 / 98,63 |
| Duo+DCD | 0 / 0,39 / 17,19 / 91,41 | 0 / 30,85 / 73,82 / 96,88 |
| **FMLM** | **9,38 / 57,03 / 79,69 / 94,92** | **39,84 / 85,15 / 92,19 / 98,83** |

- Mọi lưới hợp lệ đều duy nhất và không trùng tập huấn luyện. Ở 1024 bước, các đối chứng ngang FMLM, nên lợi thế nằm ở vùng ít bước.
- **Độ đa dạng một bước:** Self-BLEU của FMLM là 0,073 trên LM1B và 0,121 trên OWT, dữ liệu thật là 0,047 và 0,046. Trọng tài GPT-4.1 cho FMLM thắng 39 đến 42% về đa dạng. Kết luận: không sụp đổ mode, nhưng kém đa dạng hơn dữ liệu thật, rõ nhất trên OWT.

# 10. FMLM: Nghiên cứu loại bỏ thành phần

**Mô hình dòng FLM, Gen. PPL trên LM1B ở 1024 bước:**

- **Hàm mục tiêu:** vận tốc với sai số bình phương 3801,36; bộ khử nhiễu với sai số bình phương 129,04; thêm softmax 120,16; thêm cross-entropy **96,91**.
- **Biểu diễn:** one-hot 130,42, tốt hơn embedding học được (324,66), BERT-large đóng băng (262,92), embedding ngẫu nhiên (400,17).
- **Khung khuếch tán:** Euclid 96,91. Đơn hình cho PPL 85,07 nhưng entropy sụp còn 3,76. Riemann cho 268,21.

**Ánh xạ dòng FMLM, Gen. PPL một bước:**

- **Tham số hóa:** vận tốc trung bình với sai số bình phương 199,01; bộ khử nhiễu hai thời điểm với softmax và cross-entropy **119,34**.
- **Mục tiêu chưng cất:** nửa nhóm 119,34; Eulerian 149,13; Lagrangian 193,08. Chưng cất tốt hơn tự chưng cất (159,53).
- **Nhiễu đầu vào:** Gaussian 105,48 so với đều trên đơn hình 660,33.

# 11. FMLM: Hạn chế, phản biện và kết luận

**Điểm cần thận trọng khi đọc kết quả:**

- Chỉ số chính là Gen. PPL theo GPT-2 Large cộng entropy, không có đánh giá của người.
- Entropy của FMLM luôn thấp hơn dữ liệu thật: một phần lợi thế PPL có thể đến từ việc dồn mẫu về mode phổ biến.
- Quy mô nhỏ: 179M tham số. Chi phí huấn luyện hai giai đoạn không được tính vào so sánh. One-hot tốn thêm khoảng 30% thời gian và bộ nhớ.

**Kết luận:**

1. Dòng liên tục trên one-hot cạnh tranh được với khuếch tán rời rạc ở nhiều bước và vượt rõ ở ít bước.
2. Bộ khử nhiễu hai thời điểm đưa ánh xạ dòng lên đơn hình, nên chưng cất bằng cross-entropy và KL ổn định.
3. Mở đường cho dẫn hướng và học tăng cường. Chưa có bằng chứng ở quy mô LLM.

# LPT: Tổng quan bài báo

- **Tên bài báo:** *The Surprising Effectiveness of Shared Memory in Looped Transformers*
- **Tác giả và tổ chức:** Giovanni Monea, Keshav Ramji, Yousef El-Kurdi et al., Cornell và IBM Research, arXiv:2610.02383
- **Ý tưởng cốt lõi:** Huấn luyện Transformer vòng lặp từ đầu với bộ nhớ dùng chung: chỉ vòng lặp đầu ghi KV cache đầy đủ, các vòng sau đọc lại cache đó cộng một cửa sổ cục bộ ngắn.
- **Đóng góp nổi bật:**
  - **Chia sẻ bộ nhớ không làm giảm chất lượng mà còn cải thiện nó:** LPT giảm PPL 0,38 đến 1,09 so với Transformer vòng lặp thường, bộ nhớ KV nhỏ hơn 2,0 đến 4,7 lần.
  - **Mô hình lai H-LPT:** ở 5 vòng lặp, PPL thấp hơn Transformer chuẩn 1,12 đến 1,82 và dùng ít hơn 76 đến 79% bộ nhớ ngữ cảnh.
  - **Giải thích cơ chế:** đại lộ gradient, phân hóa vai trò giữa các vòng lặp.

# 1. LPT: Bối cảnh và cái bẫy bộ nhớ KV cache

**Transformer vòng lặp:** áp dụng cùng một ngăn xếp $L$ tầng $R$ lần cho mỗi token.

$$h^{(t)} = M_L\big(h_{\text{in}}^{(t)}\big), \qquad h_{\text{in}}^{(t+1)} = N_{\text{rec}}\big(h^{(t)}\big), \qquad t = 1, \ldots, R$$

- Độ sâu hiệu dụng $L \cdot R$ nhưng số tham số chỉ bằng $L$ tầng.

**Cái bẫy bộ nhớ:**

- Mỗi vòng lặp ghi một KV cache riêng, nên bộ nhớ KV tăng tuyến tính theo số vòng lặp: $M_{\text{KV}}^{\text{LT}} = L \cdot R \cdot T$.
- Ở ngữ cảnh dài, bộ nhớ ngữ cảnh có thể vượt bộ nhớ trọng số, phá hỏng lý do dùng mô hình vòng lặp.

**Giải pháp cũ:** bỏ hoặc dùng lại cache lúc suy luận, không huấn luyện cho việc đó. Chỉ giữ cache của vòng đầu làm chất lượng của Ouro giảm mạnh. Chia sẻ bộ nhớ trước đây chỉ được xem là cách tiết kiệm, chấp nhận mất chất lượng.

# 2. LPT: Kiến trúc Transformer vòng lặp dự đoán

**Tách việc xây bộ nhớ khỏi việc dự đoán theo trục vòng lặp:**

- **Vòng lặp trạng thái,** $t = 1$**:** chạy như bình thường, ghi KV cache đầy đủ $C^{(1)}$ ở mọi tầng.
- **Vòng lặp dự đoán,** $t \geq 2$**:** đọc $C^{(1)}$ cộng cửa sổ cục bộ $\omega = 64$ vị trí gần nhất của chính nó, trong cùng một phép softmax:

$$\mathcal{A}^{(t)}_{\text{LPT}}(i) = C^{(1)}_{\leq i} \cup C^{(t)}_{[i-\omega,\, i]}, \qquad t = 2, \ldots, R$$

- **Bộ nhớ KV:** $M^{\text{LPT}}_{\text{KV}} = L\,[\,T + (R-1)\omega\,]$. Với $T = 4096$, $\omega = 64$, $R = 5$: nhỏ hơn mô hình vòng lặp thường 4,7 lần.
- **Mô hình lai H-LPT:** dùng nền mô hình lai Gated DeltaNet với một tầng chú ý đầy đủ trên năm tầng. Các tầng chú ý tuyến tính giữ trạng thái lặp riêng nhỏ cố định. Các tầng chú ý đầy đủ áp dụng quy tắc trên.
- **Huấn luyện:** mặt nạ khối của FlexAttention trên khóa giá trị dùng chung nối với khóa giá trị cục bộ.

# 3. LPT: Thiết lập thực nghiệm

- **Dữ liệu và cỡ mô hình:** FineWeb-Edu, ngữ cảnh 4096. Ba cỡ: S 152M, M 454M, L 1,01B tham số, huấn luyện 20B, 20B và 50B token.
- **Số vòng lặp:** 2, 3, 5. Phụ lục thêm 9.
- **Mô hình đối chứng:** Transformer chuẩn, Transformer sâu mà hẹp khớp độ sâu hiệu dụng và số tham số, Transformer vòng lặp thường, Transformer vòng lặp song song PLT, LOOPIE, mô hình lai LT2.
- **Hai hàm mục tiêu:** mất mát ở vòng cuối, và mất mát thích ứng của Ouro có phân phối dừng học được. Transformer vòng lặp thường dùng mất mát thích ứng, các mô hình còn lại dùng mất mát ở vòng cuối.
- **Đánh giá:** PPL trên 10M token giữ lại, độ chính xác zero-shot trung bình năm tác vụ (ARC-Easy, HellaSwag, PIQA, SciQ, LAMBADA), bộ nhớ và FLOPs tính giải tích tại ngữ cảnh 4096.
- **Không điều chỉnh siêu tham số:** dùng nguyên cấu hình mặc định của nanoGPT cho mọi mô hình.

# 4. LPT: Kết quả chính ở cỡ 1B, 5 vòng lặp

| **Mô hình** | **PPL** ↓ | **Độ chính xác** ↑ | **Bộ nhớ KV so với chuẩn** ↓ |
|---|---|---|---|
| Transformer chuẩn | 11,35 | 62,5 | $1{,}00\times$ |
| Transformer vòng lặp thường | 11,39 | 63,4 | $5{,}00\times$ |
| PLT | 10,59 | 66,4 | $1{,}06\times$ |
| LT2 mô hình lai | 10,32 | 67,3 | $1{,}01\times$ |
| **LPT** | **10,32** | **68,2** | $1{,}06\times$ |
| **H-LPT** | **10,23** | 68,1 | $\mathbf{0{,}24}\times$ |

- **LPT so với Transformer vòng lặp thường, mọi cỡ và độ sâu:** PPL thấp hơn 0,38 đến 1,09, độ chính xác cao hơn 0,4 đến 4,8 điểm, bộ nhớ KV nhỏ hơn 2,0 đến 4,7 lần, FLOPs tăng dưới 1%.
- **H-LPT so với Transformer chuẩn, 5 vòng lặp:** PPL thấp hơn 1,12 đến 1,82, độ chính xác cao hơn 3,1 đến 5,6 điểm, bộ nhớ ngữ cảnh ít hơn 76 đến 79%.

# 5. LPT: Mở rộng theo độ sâu và độ ổn định huấn luyện

- **Transformer vòng lặp thường ngừng cải thiện từ 3 vòng lặp trở đi**, ở mọi cỡ. LPT tiếp tục cải thiện đến 5 vòng lặp.
- **Hàm mục tiêu ở vòng cuối:**
  - Transformer vòng lặp thường huấn luyện bằng mất mát vòng cuối thì xấu dần khi tăng vòng lặp và sụp xuống mức unigram ở 5 vòng, với cấu hình chuẩn hóa của Ouro (S: PPL 2040).
  - LPT ổn định, và mất mát vòng cuối tốt hơn mất mát thích ứng 0,16 đến 0,50 PPL: các vòng giữa được tự do tinh chỉnh trạng thái mà không phải dự đoán sớm.
- **Chín vòng lặp:** từ 5 lên 9, LPT cải thiện nhẹ ở S nhưng giảm độ chính xác 1,7 điểm ở L. Mô hình lai H-LPT tiếp tục tốt lên ở mọi cỡ.

**Ý nghĩa**

Chia sẻ bộ nhớ làm huấn luyện vòng lặp sâu ổn định hơn, nhưng không loại bỏ hoàn toàn bất ổn khi quá sâu.

# 6. LPT: Giải mã cơ chế 1 – Đại lộ gradient

**Gradient từ mất mát cuối đến vòng lặp đầu yếu đi theo độ sâu:**

- Ở Transformer vòng lặp thường, chuẩn gradient đến vòng đầu giảm mạnh khi tăng số vòng lặp.
- Ở LPT, các vòng sau đọc trực tiếp KV cache của vòng đầu, nên gradient có đường tắt về vòng đầu. Ở 5 vòng lặp, phần lớn gradient đến vòng đầu đi qua đường này.

**Bằng chứng nhân quả: cắt gradient của KV cache dùng chung.**

- Hướng thuận không đổi, chỉ cắt đường gradient. PPL tăng ở mọi cấu hình. Cỡ S, 5 vòng: 14,91 tăng lên 33,21. Cỡ M, 3 và 5 vòng: không huấn luyện được, PPL gần mức unigram.

**Điểm cần thận trọng:**

- Tỉ lệ gradient qua KV dùng chung so với gradient qua trạng thái ẩn đã tăng theo độ sâu ngay từ lúc khởi tạo, nên xu hướng này tự nó chưa chứng minh lợi ích.
- Tác giả thừa nhận thí nghiệm chưa phân biệt được đường gradient ngắn hơn với thay đổi độ lớn hoặc hướng gradient.

# 7. LPT: Giải mã cơ chế 2 – Phân hóa vai trò và cửa sổ cục bộ

**Các vòng dự đoán chú ý chủ yếu vào bộ nhớ dùng chung:**

- Tỉ lệ chú ý vào bộ nhớ dùng chung: 57 đến 71% ở các vòng dự đoán đầu, 73 đến 80% ở vòng cuối, kể cả khi cửa sổ cục bộ phủ toàn ngữ cảnh.

**Khóa dùng chung và khóa cục bộ tách thành các cụm riêng trong PCA:**

- Ở Transformer vòng lặp thường, khóa của các vòng chồng lên nhau nhiều hơn.
- Vậy sự giống nhau giữa các vòng không phải điều kiện cần để chia sẻ bộ nhớ hiệu quả: vòng đầu chuyên xây bộ nhớ, các vòng sau chuyên dùng nó.
- Đây là bằng chứng định tính, trên một chuỗi kiểm tra.

**Quét cửa sổ $\omega$ từ 0 đến 4096:**

- LPT tốt hơn Transformer vòng lặp thường ở mọi $\omega$. Cửa sổ ảnh hưởng ít, điều quyết định là được đọc bộ nhớ dùng chung. Chọn $\omega = 64$.
- Lưu ý: ở $\omega = 0$, vòng dự đoán vẫn chú ý vào mục nhập hiện tại của chính nó.

# 8. LPT: Đo thực tế trên GPU H100

**Bộ nhớ nhỏ hơn cho phép batch lớn hơn.** Ngữ cảnh 4096, bf16, một mã nguồn nghiên cứu dùng chung cho mọi mô hình.

- **Sức chứa batch:** LPT chứa 93 đến 98% batch của Transformer chuẩn, và gấp 2,0; 2,9; 4,7 đến 4,8 lần Transformer vòng lặp thường ở 2, 3, 5 vòng lặp. H-LPT chứa gấp 3,6 đến 5,0 lần Transformer chuẩn.
- **Thông lượng của H-LPT so với Transformer chuẩn, tại batch tối đa:** 2,1 đến 2,2 lần ở 2 vòng; 1,4 đến 1,5 lần ở 3 vòng; 0,8 đến 0,9 lần ở 5 vòng.

**Chi phí không biến mất:**

- Cùng batch, LPT chậm hơn Transformer vòng lặp thường 3 đến 8%, vì vẫn phải đọc cả cache dùng chung. Bộ nhớ dùng chung tiết kiệm dung lượng, không tiết kiệm thời gian đọc.
- Batch bằng 1: một bước giải mã của Transformer vòng lặp thường chậm hơn chuẩn 2,3; 3,4; 5,7 lần ở 2, 3, 5 vòng. Ở 5 vòng LPT tốn 4,3 đến 4,8 lần FLOPs của Transformer chuẩn.

# 9. LPT: Độ tin cậy thống kê và tính công bằng

**Bốn hạt giống ngẫu nhiên ở cỡ S, kiểm định t ghép cặp một phía:**

| Chênh lệch PPL | 2 vòng | 3 vòng | 5 vòng |
|---|---|---|---|
| LPT so với Transformer vòng lặp thường | $-0{,}40$ | $-0{,}71$ | $-1{,}05$ |
| H-LPT so với LT2 | $-0{,}04$ | $-0{,}07$ | $-0{,}13$ |
| LPT so với PLT | $-0{,}07$ | $-0{,}27$ | $-0{,}52$ |

- Độ lệch chuẩn giữa các hạt giống lớn nhất là 0,08 PPL. Lợi thế thật sự lớn chỉ khi từ 3 vòng trở lên. Ở 2 vòng, LPT so với PLT gần hòa.
- **Phép so sánh bị lệch có lợi cho LPT:** cấu hình chuẩn hóa của Ouro làm Transformer vòng lặp thường yếu. Thử thêm 5 cách chuẩn hóa khác, bản tốt nhất giảm PPL của nó 0,13 đến 0,76. Khi đó khoảng cách còn **0,31 đến 0,37** ở S và **0,06 đến 0,43** ở M.
- Chỉ cỡ S có nhiều hạt giống. Cỡ M và L là một lần chạy.

# 10. LPT: Hạn chế, phản biện và kết luận

**Hạn chế:**

- Không tối ưu về tính toán: 5 vòng lặp tốn 4,3 đến 4,8 lần FLOPs giải mã. Đối chứng khớp tham số, không khớp FLOPs. Mọi token dùng cùng số vòng lặp.
- Tối đa 1B tham số, một kho dữ liệu, ngữ cảnh 4096, chưa thử ngữ cảnh dài. Chênh độ chính xác dưới 1 điểm nên xem là nhiễu.
- Chưa tối ưu siêu tham số, chưa công bố mã nguồn.

**Kết luận:**

- Chia sẻ bộ nhớ là một thiên kiến quy nạp tốt cho mô hình vòng lặp, không chỉ là mẹo tiết kiệm bộ nhớ.
- H-LPT tạo biên chất lượng và bộ nhớ mới cho mô hình vòng lặp: PPL thấp hơn Transformer chuẩn với ít hơn 76 đến 79% bộ nhớ ngữ cảnh.
- **Hướng tiếp theo:** số vòng lặp thích ứng theo token, nhiều vòng trạng thái hoặc xen kẽ vòng trạng thái và vòng dự đoán, ngữ cảnh dài.

# Cảm ơn bạn đã theo dõi!

Q&A